Perceptrón: clasificación lineal binaria y clasificación de cuatro clases con dos perceptrones

1st Oscar Alberto Gallo García
M. C. en Robótica e Inteligencia Artificial
Centro Universitario de Ciencias Exactas e Ingenierías
Guadalajara, México
oscar.gallo5770@alumnos.udg.mx

***Resumen*—Se implementó desde cero el algoritmo de aprendizaje del perceptrón de Rosenblatt y se aplicó a dos problemas linealmente separables. En el primero, un perceptrón de tres entradas aprendió la función lógica $f(x_1, x_2, x_3) = (x_1 \wedge x_2) \vee (x_1 \wedge x_3) \vee (x_2 \wedge x_3)$, alcanzando error de clasificación cero en 6 épocas; el plano de decisión obtenido separa correctamente los ocho vértices del cubo. En la segunda parte de la práctica, dos perceptrones independientes entrenados sobre un código binario de dos bits clasificaron 16 patrones bidimensionales en cuatro clases, con exactitud del $100\,\%$ en entrenamiento tras 4 y 3 épocas. Sobre ocho patrones de prueba no vistos, la red asignó clases consistentes con las regiones aprendidas; sin embargo, cuatro de ellos quedaron a menos de $0{,}08$ unidades de una frontera y uno a solo $0{,}0009$, lo que muestra que el perceptrón no maximiza el margen y que su generalización cerca de las fronteras es frágil.**

***Index Terms*—Perceptrón, neurona artificial, clasificación lineal, linealmente separable, generalización**

I. Introducción

El perceptrón, propuesto por Rosenblatt en 1958 [4], es la neurona artificial más simple capaz de aprender: calcula una combinación lineal de sus entradas, la compara con un umbral y corrige sus pesos únicamente cuando se equivoca [1], [2]. Su regla de aprendizaje tiene una garantía fuerte: si las dos clases son linealmente separables, el algoritmo converge en un número finito de correcciones (teorema de convergencia del perceptrón) [1], [3]. Cuando no lo son, como en la función XOR, el perceptrón nunca alcanza error cero, limitación señalada por Minsky y Papert [5] que motivó las redes multicapa [2].

El objetivo de esta práctica es implementar y analizar el algoritmo del perceptrón en dos problemas linealmente separables: (i) aprender una función lógica de tres variables con una sola neurona e interpretar geométricamente el plano de decisión; y (ii) construir un clasificador de cuatro clases en el plano con dos perceptrones, obtener la matriz de confusión y evaluar la capacidad de generalización sobre patrones no usados en el entrenamiento.

II. Desarrollo

II-A. Modelo del perceptrón

Para un patrón $\mathbf{x} \in \mathbb{R}^n$ la neurona calcula el campo local inducido

$$v = \mathbf{w}^\top \mathbf{x} + b = \sum_{i=1}^{n} w_i x_i + b, \tag{1}$$

donde $\mathbf{w}$ es el vector de pesos y b el sesgo (*bias*), y produce la salida binaria con la función escalón

$$y = \varphi(v) = \begin{cases} 1, & v \geq 0 \\ 0, & v < 0. \end{cases} \tag{2}$$

La frontera de decisión es el hiperplano $\mathbf{w}^\top \mathbf{x} + b = 0$: un plano en $\mathbb{R}^3$ para la primera práctica y una recta en $\mathbb{R}^2$ para la segunda. El vector $\mathbf{w}$ es normal a ese hiperplano y apunta hacia la región donde $y = 1$.

II-B. Regla de aprendizaje

Se usó la regla de corrección de error de Rosenblatt [1]. Para cada patrón k con salida deseada $d(k) \in \{0, 1\}$ se calcula $y(k)$ con (1)–(2) y se actualiza

$$\mathbf{w}(k+1) = \mathbf{w}(k) + \eta\,[d(k) - y(k)]\,\mathbf{x}(k), \tag{3}$$

$$b(k+1) = b(k) + \eta\,[d(k) - y(k)], \tag{4}$$

con $\eta > 0$ la razón de aprendizaje. Como $d - y \in \{-1, 0, +1\}$, el patrón bien clasificado no modifica nada; un falso negativo ($d = 1, y = 0$) suma $\eta\mathbf{x}$ a los pesos y acerca la frontera a la región correcta, y un falso positivo ($d = 0, y = 1$) resta $\eta\mathbf{x}$. A diferencia de ADALINE, el error se calcula con la salida *cuantizada* y, no con v, por lo que la regla no proviene del gradiente de una función de costo continua, sino de un argumento geométrico de corrección.

II-C. Algoritmo e implementación

El algoritmo se programó en Python con NumPy para las operaciones vectoriales y Matplotlib para las gráficas, sin usar ninguna implementación preconstruida del perceptrón. Las actualizaciones son *en línea*: se aplican inmediatamente después de cada patrón. El criterio de paro es error de clasificación igual a cero en una pasada completa por el conjunto de entrenamiento; como medida de seguridad se fijó un máximo de 100 épocas, que nunca se alcanzó. Los pesos y sesgos iniciales se obtuvieron de una distribución uniforme $\mathcal{U}(0, 1)$ con semilla 0 del generador de NumPy para que los resultados sean reproducibles.

1: Inicializar $\mathbf{w}(0)$, $b(0) \sim \mathcal{U}(0, 1)$
2: **for** época $= 1$ **to** 100 **do**
3: errores $\leftarrow 0$
4: **for** cada patrón k en orden fijo **do**

```
      v ← w^⊤ x(k) + b;   y ← [v ≥ 0]
      if y ≠ d(k) then
        w ← w + η (d(k) − y) x(k);   b ← b + η (d(k) − y)
        errores ← errores + 1
      end if
    end for
    if errores = 0 then
      Detener
    end if
  end for
```

Para la segunda práctica el mismo procedimiento se encapsuló en una clase `perceptron` con métodos de entrenamiento y verificación, de la que se instanciaron dos objetos independientes.

II-D. Práctica 1: función lógica de tres variables

La función $f(x_1, x_2, x_3) = (x_1 \wedge x_2) \vee (x_1 \wedge x_3) \vee (x_2 \wedge x_3)$ vale 1 cuando al menos dos de las tres entradas valen 1, es decir, es la función de *mayoría*. Su tabla de verdad (Cuadro I) define los ocho patrones de entrenamiento, que son los vértices del cubo unitario, y las clases C_0 ($f = 0$, cuatro patrones) y C_1 ($f = 1$, cuatro patrones). Se entrenó un perceptrón de tres entradas con $\eta = 0{,}5$.

Cuadro I
TABLA DE VERDAD DE $f(x_1, x_2, x_3)$ Y CLASE DE CADA PATRÓN

x_1	x_2	x_3	$x_1 \wedge x_2$	$x_1 \wedge x_3$	$x_2 \wedge x_3$	f	Clase
0	0	0	0	0	0	0	C_0
0	0	1	0	0	0	0	C_0
0	1	0	0	0	0	0	C_0
0	1	1	0	0	1	1	C_1
1	0	0	0	0	0	0	C_0
1	0	1	0	1	0	1	C_1
1	1	0	1	0	0	1	C_1
1	1	1	1	1	1	1	C_1

II-E. Práctica 2: cuatro clases con dos perceptrones

Se usaron los 16 patrones bidimensionales de la práctica, cuatro por clase, codificando cada clase con dos bits: $C_1 = (0,0)$, $C_2 = (0,1)$, $C_3 = (1,0)$ y $C_4 = (1,1)$. Dos perceptrones P_1 y P_2 con las mismas entradas (x_1, x_2) se entrenaron de forma independiente con $\eta = 0{,}1$: P_1 con la columna y_1 como objetivo (clases $C_3 \cup C_4$ contra $C_1 \cup C_2$) y P_2 con la columna y_2 ($C_2 \cup C_4$ contra $C_1 \cup C_3$). La clase final se obtiene decodificando el par $(\hat{y}_1, \hat{y}_2)$ con la tabla anterior, lo que equivale a calcular el índice $2\hat{y}_1 + \hat{y}_2$. Posteriormente se clasificaron ocho patrones de prueba usando solo los pesos y sesgos aprendidos, sin volver a entrenar.

III. RESULTADOS DE LA PRÁCTICA 1

III-A. Separabilidad lineal

La Fig. 1 muestra los ocho vértices del cubo coloreados por clase. Los patrones de C_1 son los que tienen dos o tres unos, y los de C_0 los que tienen cero o un uno. Se observa que ambos grupos pueden separarse con un plano que corte el cubo entre la "capa" de vértices con un uno y la de vértices con dos unos. Analíticamente, el plano $x_1 + x_2 + x_3 = 1{,}5$ realiza esa separación, por lo que las clases son linealmente separables y se espera convergencia del perceptrón.

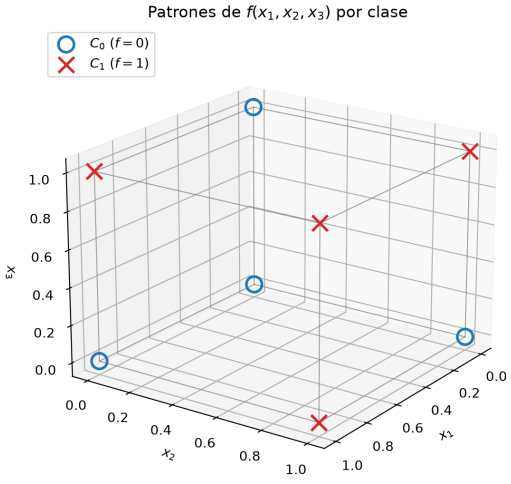


Figura 1. Patrones de la función f en el espacio (x_1, x_2, x_3). Círculos azules: C_0 ($f = 0$); cruces rojas: C_1 ($f = 1$). Las aristas grises unen vértices adyacentes del cubo.

III-B. Entrenamiento

Los parámetros iniciales fueron $\mathbf{w}(0) = [0{,}5488,\ 0{,}7152,\ 0{,}6028]^\top$ y $b(0) = 0{,}5449$. Con estos valores todos los campos locales son positivos y el perceptrón inicial clasifica todo como C_1. El Cuadro II y la Fig. 2 muestran la evolución por época. En la primera época se corrigen los tres patrones de C_0 distintos del origen que se recorren antes del origen; el número de errores oscila entre 2 y 3 durante cinco épocas y cae a cero en la sexta. En total hubo 12 correcciones.

Cuadro II
EVOLUCIÓN DEL ENTRENAMIENTO DEL PERCEPTRÓN DE TRES ENTRADAS ($\eta = 0{,}5$)

Época	Errores	w_1	w_2	w_3
0	–	0,5488	0,7152	0,6028
1	3	0,0488	0,2152	0,1028
2	2	0,0488	0,7152	0,6028
3	3	0,0488	0,7152	0,6028
4	2	0,5488	0,7152	1,1028
5	2	0,5488	1,2152	1,1028
6	0	0,5488	1,2152	1,1028

El sesgo evolucionó como $b = 0{,}5449, -0{,}9551, -0{,}9551, -1{,}4551$ y permaneció en $-1{,}4551$ desde la época 3.

III-C. Pesos finales y plano de decisión

Los parámetros finales son

$$\mathbf{w} = \begin{bmatrix} 0{,}5488 & 1{,}2152 & 1{,}1028 \end{bmatrix}^\top, \qquad b = -1{,}4551, \quad (5)$$

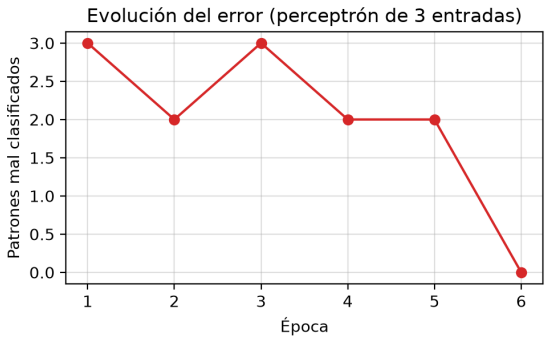


Figura 2. Número de patrones mal clasificados por época (Práctica 1).

de modo que el plano de decisión es $0{,}5488x_1 + 1{,}2152x_2 + 1{,}1028x_3 = 1{,}4551$. La Fig. 3 lo muestra junto con los patrones: los cuatro vértices de C_1 quedan del lado positivo del plano y los cuatro de C_0 del lado negativo. El Cuadro III confirma la clasificación correcta de los ocho patrones a partir del campo local v.

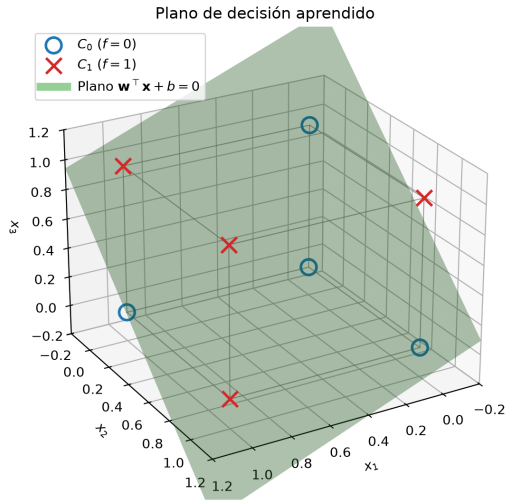


Figura 3. Plano de decisión aprendido (verde) y patrones por clase. El plano corta el cubo separando los vértices con dos o más unos de los demás.

Cuadro III
VERIFICACIÓN DE LOS OCHO PATRONES CON LOS PESOS FINALES

x_1	x_2	x_3	v	y	d
1	1	1	1,412	1	1
1	1	0	0,309	1	1
1	0	1	0,196	1	1
1	0	0	−0,906	0	0
0	1	1	0,863	1	1
0	1	0	−0,240	0	0
0	0	1	−0,352	0	0
0	0	0	−1,455	0	0

III-D. Relación entre la función lógica y la frontera

La función de mayoría es una *función de umbral*: $f = 1$ si $x_1 + x_2 + x_3 \geq 2$. El perceptrón aprendió exactamente esa estructura. Todos los pesos son positivos, por lo que cada entrada activa acerca al patrón al lado C_1, y el sesgo negativo ($-1{,}455$) actúa como umbral. Ninguna entrada por sí sola supera el umbral ($w_i < 1{,}455$ para toda i), pero cualquier par sí lo hace: la suma mínima de dos pesos es $w_1 + w_3 = 1{,}652 > 1{,}455$. Esto es precisamente la definición de la función: $f = 1$ si y solo si al menos dos entradas están activas. Los pesos no son iguales entre sí, como lo serían en la solución "simétrica" $x_1 + x_2 + x_3 = 1{,}5$, porque el perceptrón se detiene en la primera solución que alcanza error cero, la cual depende de la inicialización y del orden de presentación. El patrón $(1, 0, 1)$ queda a solo $v = 0{,}196$ del plano, el margen más pequeño de la solución.

IV. RESULTADOS DE LA PRÁCTICA 2

IV-A. Conjunto de entrenamiento

La Fig. 4 muestra los 16 patrones. Las clases ocupan aproximadamente cuatro sectores del plano: C_1 a la izquierda, C_2 abajo, C_3 arriba y C_4 a la derecha. La unión $C_3 \cup C_4$ (objetivo de P_1) se separa de $C_1 \cup C_2$ con una recta de pendiente negativa, y $C_2 \cup C_4$ (objetivo de P_2) se separa de $C_1 \cup C_3$ con una recta de pendiente positiva, por lo que ambos subproblemas binarios son linealmente separables.

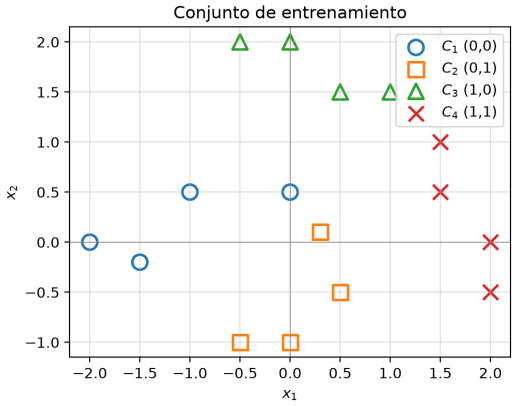


Figura 4. Conjunto de entrenamiento de la Práctica 2, con un marcador por clase y su código (y_1, y_2).

IV-B. Entrenamiento y parámetros finales

El Cuadro IV reporta los parámetros iniciales y finales de ambos perceptrones y la Fig. 5 la evolución del error. P_1 convergió en 4 épocas ($4, 3, 2, 0$ errores) y P_2 en 3 épocas ($6, 2, 0$ errores). En ambos casos la inicialización positiva hace que al inicio casi todos los patrones se clasifiquen como 1, y las correcciones sobre los patrones de la clase 0 van rotando y desplazando la recta hasta separar los grupos.

Cuadro IV
PARÁMETROS DE LOS DOS PERCEPTRONES ($\eta = 0{,}1$)

	$w_1(0)$	$w_2(0)$	$b(0)$	Épocas
P_1	0,5488	0,7152	0,6028	4
P_2	0,5449	0,4237	0,6459	3
	w_1	w_2	b	Recta $x_2 = mx_1 + c$
P_1	0,4588	0,5852	$-0{,}2972$	$x_2 = -0{,}784x_1 + 0{,}508$
P_2	0,3449	$-0{,}4763$	0,0459	$x_2 = 0{,}724x_1 + 0{,}096$

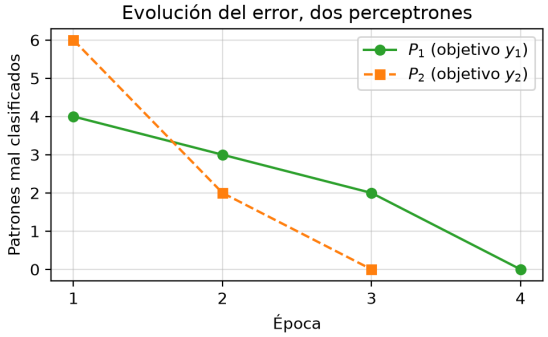


Figura 5. Patrones mal clasificados por época para P_1 y P_2.

IV-C. Fronteras de decisión y regiones

La Fig. 6 muestra las dos rectas aprendidas y las cuatro regiones que generan. La frontera de P_1 (verde) tiene pendiente negativa y deja arriba-derecha a C_3 y C_4 ($y_1 = 1$); la de P_2 (naranja) tiene pendiente positiva y deja abajo-derecha a C_2 y C_4 ($y_2 = 1$). Ambas rectas se cruzan en $(0{,}273,\ 0{,}294)$ y dividen el plano en cuatro regiones angulares, cada una asociada a un código distinto: izquierda $(0,0) = C_1$, abajo $(0,1) = C_2$, arriba $(1,0) = C_3$ y derecha $(1,1) = C_4$. Cada perceptrón responde una pregunta binaria sobre la posición del patrón y la combinación de las dos respuestas identifica el sector.

Es notable que la recta de P_1 pasa casi por el patrón $(0,\ 0{,}5)$ de C_1: su campo local es $v = -0{,}0046$ y su distancia a la frontera es de solo $0{,}006$ unidades. El perceptrón lo clasifica bien, pero sin margen. La distancia mínima de los patrones de entrenamiento a la recta de P_2 es mayor, $0{,}148$.

IV-D. Matriz de confusión y exactitud

Al clasificar los 16 patrones de entrenamiento con los pesos finales y el decodificador, los 16 quedaron en la diagonal de la matriz de confusión (Fig. 7): cuatro aciertos por clase y ningún error, es decir, exactitud del $100\,\%$. Esto era esperable: cada perceptrón alcanzó error cero sobre su bit, y si ambos bits son correctos la clase decodificada también lo es.

V. PRUEBAS Y ANÁLISIS DE GENERALIZACIÓN

Los ocho patrones de prueba se clasificaron únicamente con los pesos aprendidos. El Cuadro V muestra las salidas de cada perceptrón, la clase predicha y la distancia con signo de cada

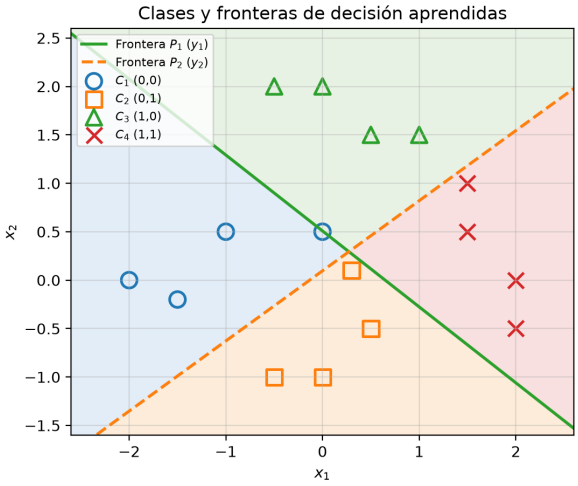


Figura 6. Cuatro clases y las dos fronteras aprendidas. Las zonas sombreadas son las regiones que el decodificador asigna a cada clase.

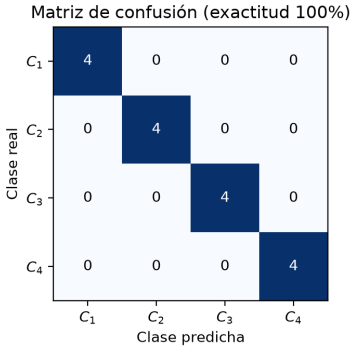


Figura 7. Matriz de confusión 4×4 sobre el conjunto de entrenamiento.

patrón a cada frontera, $\delta_j = (\mathbf{w}_j^\top \mathbf{x} + b_j)/\|\mathbf{w}_j\|$, donde el signo indica el lado de la recta y la magnitud qué tan lejos está. La Fig. 8 ubica los patrones de prueba sobre las regiones aprendidas y la Fig. 9 amplía la zona de cruce de las fronteras.

Cuadro V
CLASIFICACIÓN DE LOS PATRONES DE PRUEBA

#	x_1	x_2	$\hat{y}_1$	$\hat{y}_2$	δ_1	δ_2	Clase
1	$-1{,}20$	0,70	0	0	$-0{,}589$	$-1{,}193$	C_1
2	0,45	0,50	1	0	0,271	$-0{,}063$	C_3
3	0,20	$-0{,}80$	0	1	$-0{,}906$	0,843	C_2
4	0,49	0,45	1	1	0,257	0,0009	C_4
5	$-0{,}20$	1,50	1	0	0,657	$-1{,}254$	C_3
6	0,49	0,55	1	0	0,335	$-0{,}080$	C_3
7	1,60	0,20	1	1	0,745	0,854	C_4
8	0,55	0,49	1	1	0,325	0,0037	C_4

Patrones próximos a una frontera. Los patrones 2, 4, 6 y 8 están a menos de $0{,}08$ unidades de la recta de P_2 (Cuadro V, Fig. 9). Los casos extremos son el patrón 4, a

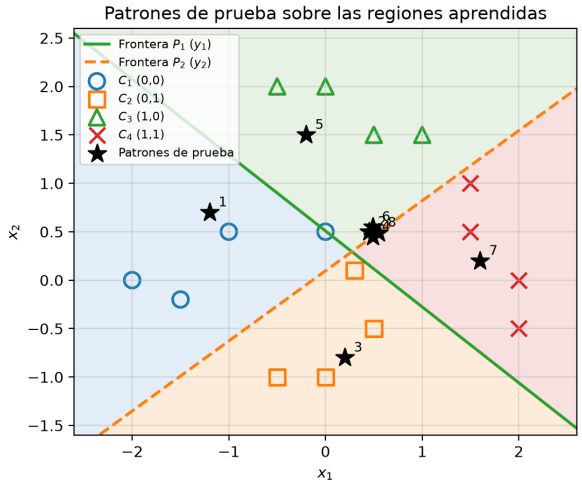


Figura 8. Patrones de prueba (estrellas numeradas) sobre las regiones aprendidas.

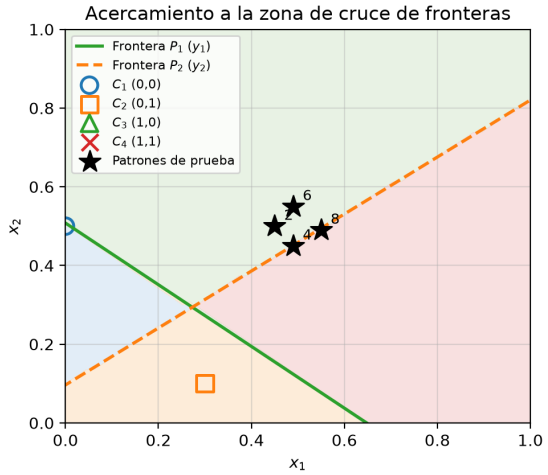


Figura 9. Acercamiento a la zona $[0,1] \times [0,1]$: los patrones 2, 4, 6 y 8 se encuentran junto a la frontera de P_2.

0,0009 unidades, y el 8, a 0,0037. Estos cuatro patrones fueron colocados deliberadamente alrededor de $(0{,}5, 0{,}5)$, a ambos lados de la frontera que separa C_3 de C_4. Los patrones 1, 3, 5 y 7, en cambio, están a más de 0,58 unidades de ambas fronteras, en el interior de las regiones.

Patrones alejados contra patrones cercanos. Para los patrones alejados la predicción es robusta y coincide con lo que sugiere la distribución del entrenamiento: el patrón 1 está entre los círculos de C_1, el 3 entre los cuadrados de C_2, el 5 entre los triángulos de C_3 y el 7 entre las cruces de C_4. Para los patrones cercanos la decisión depende de milésimas: los patrones 4 $(0{,}49, 0{,}45)$ y 6 $(0{,}49, 0{,}55)$ difieren solo en 0,1 en x_2 y reciben clases distintas (C_4 y C_3), y lo mismo ocurre con los patrones 2 y 8. En esa zona no hay patrones de entrenamiento de C_3 ni de C_4 (los más cercanos están en $(0{,}5, 1{,}5)$ y $(1{,}5, 0{,}5)$), por lo que la posición exacta de la frontera allí no está respaldada por datos sino por el azar de la inicialización y del orden de las correcciones.

Efecto de una perturbación. Una perturbación Δ en el patrón cambia δ_j en a lo sumo $\|\Delta\|$. Para el patrón 4 basta $\|\Delta\| > 0{,}0009$ en la dirección normal a la recta de P_2, por ejemplo $x_2 = 0{,}451$ en lugar de 0,450, para que $\hat{y}_2$ pase de 1 a 0 y la clase cambie de C_4 a C_3; para el patrón 8 basta 0,004 y para los patrones 2 y 6 cerca de 0,06–0,08. En cambio el patrón 7 tolera perturbaciones de hasta 0,74 unidades sin cambiar de clase. Como además la frontera de P_1 pasa a 0,006 del patrón de entrenamiento $(0, 0{,}5)$, una perturbación de ese orden en un patrón similar también cambiaría su clase de C_1 a C_3.

Significado de generalizar. En este problema generalizar es asignar a un punto nuevo la clase del sector del plano en que cae, sin haberlo visto durante el entrenamiento. Las Figs. 6 y 8 muestran que las fronteras aprendidas extienden a todo el plano la separación observada en solo 16 puntos. La generalización es buena lejos de las fronteras, donde cualquier recta separadora razonable daría la misma respuesta, y es incierta cerca de ellas, donde rectas igualmente válidas sobre el entrenamiento producirían clases distintas. Como ninguno de los patrones de prueba tiene etiqueta verdadera, no puede calcularse una exactitud de prueba; lo que sí se puede afirmar es que las predicciones para los patrones 1, 3, 5 y 7 son confiables y las de 2, 4, 6 y 8 dependen de la solución particular encontrada.

VI. Discusión

Convergencia. En ambas prácticas el perceptrón alcanzó error cero en pocas épocas (6, 4 y 3), de acuerdo con el teorema de convergencia para clases linealmente separables [1]. El número de correcciones es pequeño porque los problemas tienen pocos patrones y margen geométrico razonable; el valor de η solo escala la magnitud de cada corrección y, con inicialización aleatoria, afecta la solución final pero no la convergencia.

La solución no es única. El perceptrón se detiene en la *primera* solución con error cero, que depende de $\mathbf{w}(0)$, $b(0)$, η y del orden de los patrones. Esto explica que los pesos de la Práctica 1 no sean simétricos aunque la función de mayoría lo sea, y que en la Práctica 2 la recta de P_1 pase a 0,006 de un patrón de entrenamiento. El algoritmo no tiene ningún mecanismo para preferir la recta de margen máximo entre todas las que separan los datos; esa es la idea central de las máquinas de vectores de soporte [1], [3].

Dos clasificadores binarios para cuatro clases. Con k perceptrones de salida binaria se pueden codificar hasta 2^k clases, siempre que cada bit del código corresponda a una partición linealmente separable del conjunto. Aquí la codificación funcionó porque las uniones $C_3 \cup C_4$ y $C_2 \cup C_4$ son separables. Con otra asignación de códigos, por ejemplo si P_1 tuviera que separar $C_1 \cup C_4$ de $C_2 \cup C_3$ (sectores opuestos), el subproblema sería del tipo XOR y el perceptrón

no convergería. La codificación de las clases es, por tanto, parte del diseño.

Limitaciones.

- Sin un conjunto de prueba etiquetado no es posible medir cuantitativamente la generalización; solo puede analizarse geométricamente.
- La salida escalón no entrega una medida de confianza; usamos la distancia a la frontera como sustituto, pero el modelo no la aprende ni la optimiza.
- Las regiones aprendidas son no acotadas: un punto muy lejano, como $(10, 10)$, se asigna a C_4 aunque no se parezca a ningún patrón de entrenamiento.
- Los resultados numéricos dependen de la semilla; con otra semilla las fronteras cambian y los patrones de prueba 2, 4, 6 y 8 podrían recibir clases distintas.

VII. Conclusiones

Se implementó desde cero el perceptrón de Rosenblatt con la regla de corrección de error y se verificó experimentalmente su convergencia en dos problemas linealmente separables. En la Práctica 1 una sola neurona aprendió la función de mayoría de tres variables en 6 épocas; los pesos positivos y el sesgo negativo reproducen la estructura de umbral de la función, y el plano aprendido separa los vértices del cubo con dos o más unos de los demás. En la Práctica 2 dos perceptrones independientes dividieron el plano en cuatro regiones angulares y clasificaron correctamente los 16 patrones de entrenamiento, con exactitud del $100\,\%$ y matriz de confusión diagonal.

La prueba de generalización mostró la principal debilidad del método: el perceptrón encuentra *una* frontera separadora, no la mejor. Cuatro de los ocho patrones de prueba quedaron a menos de $0{,}08$ unidades de una frontera y uno a $0{,}0009$, de modo que perturbaciones mínimas cambian su clase. Generalizar bien en este problema requiere fronteras alejadas de los datos, lo que motiva criterios de margen máximo o funciones de costo continuas como la de ADALINE, que se estudia en la siguiente práctica.

Referencias

[1] S. Haykin, *Neural Networks and Learning Machines*, 3rd ed. Upper Saddle River, NJ, USA: Pearson/Prentice Hall, 2009.

[2] C. C. Aggarwal, *Neural Networks and Deep Learning: A Textbook*. Cham, Switzerland: Springer, 2018.

[3] B. Mehlig, *Machine Learning with Neural Networks: An Introduction for Scientists and Engineers*. Cambridge, U.K.: Cambridge University Press, 2021.

[4] F. Rosenblatt, "The perceptron: A probabilistic model for information storage and organization in the brain," *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958.

[5] M. Minsky and S. Papert, *Perceptrons: An Introduction to Computational Geometry*. Cambridge, MA, USA: MIT Press, 1969.